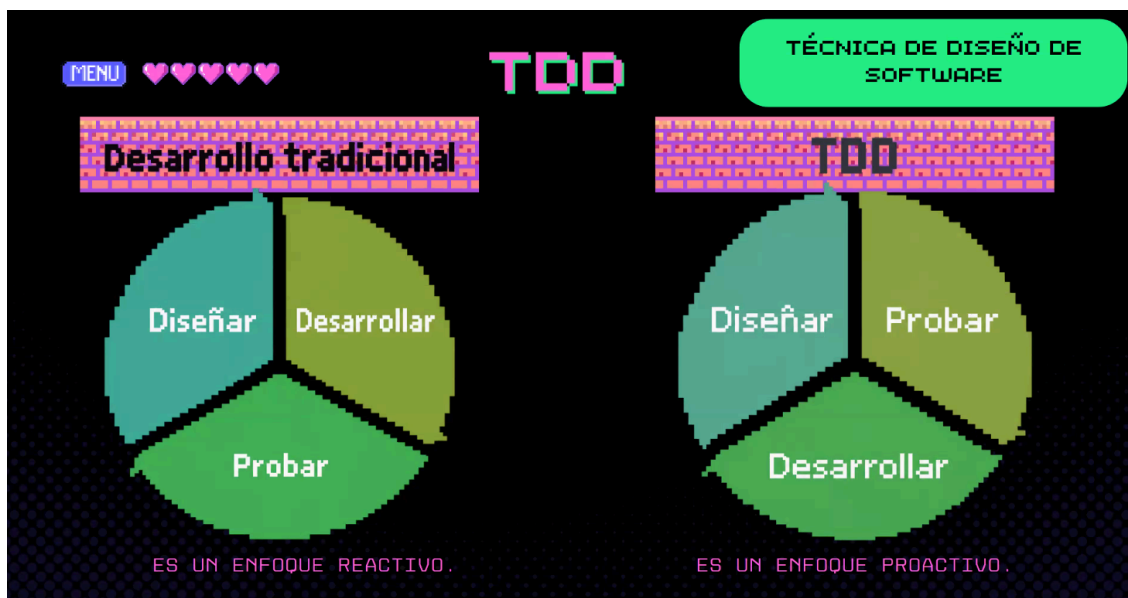


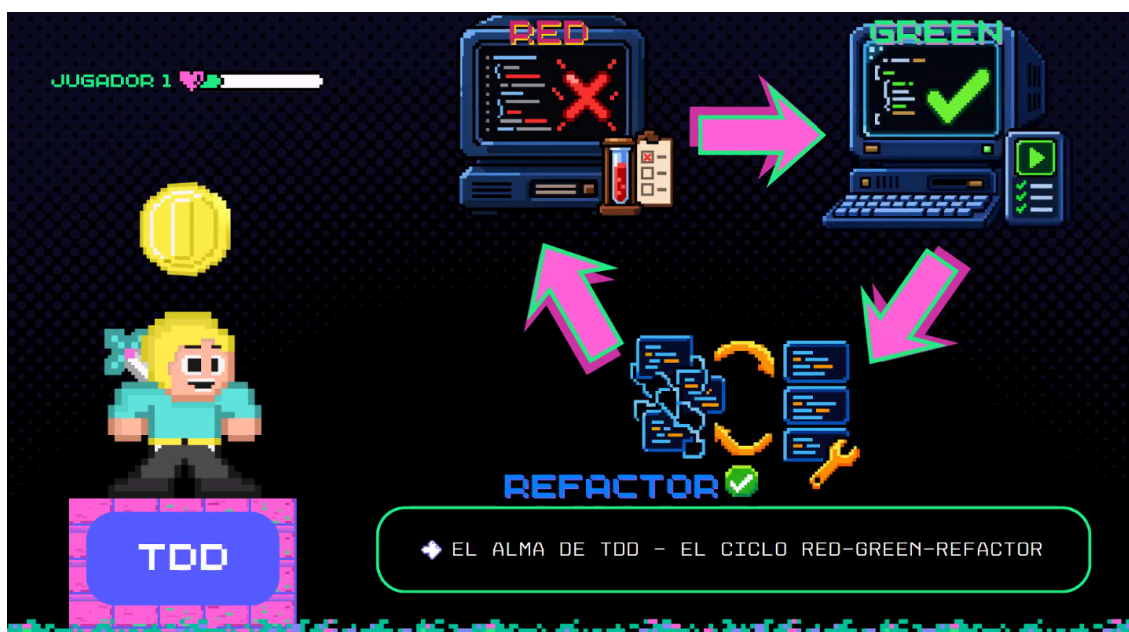
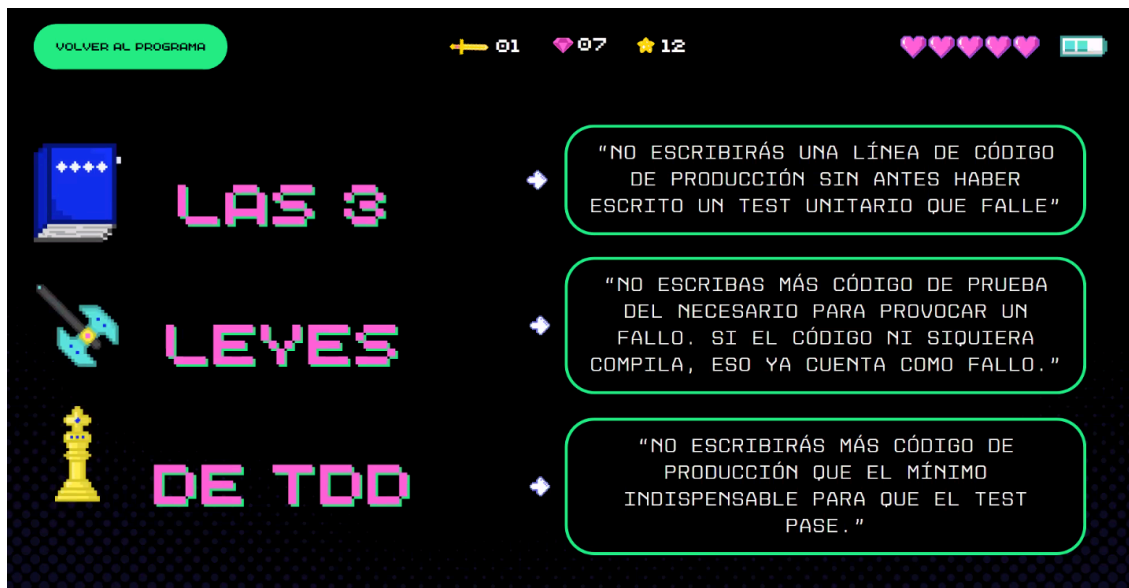
TDD

Enfoque tradicional: se hablaba del testing al final, post requerimientos, ya implementado todo. Trae problemas/complejidades a la hora de desarrollar el sistema, como el acoplamiento incidental: los desarrolladores diseñaban el código esperando que funcione, por lo que no había partes aisladas, siendo más tedioso todo a la hora de probarlo. Sumado a esto, se tiene el problema de la duda técnica y el miedo a cambio: implicaba retrabajo. También, el costo asintótico del error (el costo va incrementando a medida que pasa el tiempo, no es lo mismo).

- Acoplamiento incidental
- Deuda técnica y miedo al cambio
- Costo asintótico del error.



Se cambia el orden del desarrollar y probar.



RED

Trabajamos de manera declarativa

DECLARAMOS LA INTENCIÓN:
"QUIERO QUE AL PASARLE **ESTOS**
DATOS, EL SISTEMA ME
DEVUELVA **ESTE** RESULTADO"

```
def test_sumar_dos_numeros_EXITO():  
    //Precondiciones  
    calculadoraDePrueba = new Calculadora()  
    //Pasos  
    resultado = calculadoraDePrueba.sumar(a = 1, b = 2)  
    //Resultado  
    assert(resultado == 3)  
  
Class Calculadora:  
    def sumar(a,b):  
        return "No implementado" //Puede ser pass  
        directamente
```



Se escribe una prueba que falle. Buscamos destrabarnos a la hora de pensar que hacer, para luego en la etapa green lo resolvamos para que pase esa prueba.

En el green, escribimos el código mínimo para que pase la prueba. En el ejemplo de la calculadora, podría ser `return 3` o `return a + b`. Se usa el patrón fake it (hardcodear el resultado para que nos devuelva el pasa), el código mínimo para que pase es `return 3`

GREEN

PASAMOS A UN ENFOQUE
IMPERATIVO [CÓMO].
ESCRIBIMOS EL CÓDIGO **MINIMO**
PARA QUE LA PRUEBA PASE.

```
def test_sumar_dos_numeros_EXITO():  
    //Precondiciones  
    calculadoraDePrueba = new Calculadora()  
    //Resultados  
    resultado = calculadoraDePrueba.sumar(a = 1, b = 2)  
    assert(resultado == 3)  
  
Class Calculadora:  
    def sumar(a,b):  
        return 3
```



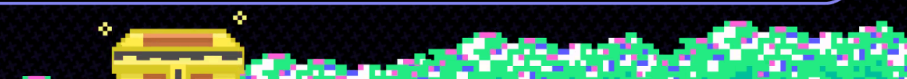
En refactor, modificamos la estructura interna, a nivel de buenas prácticas, no solo a nivel de lógica, porque podría repercutir en todo mi código.

REFACTOR

REFACTORIZAR ES MODIFICAR LA ESTRUCTURA INTERNA DEL SOFTWARE **SIN ALTERAR SU COMPORTAMIENTO EXTERNO**.

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 1,b = 2)
    //Resultados
    assert(resultado == 3)

Class Calculadora:
    def sumar(a: Number,b: Number) -> Number:
        return 3
```

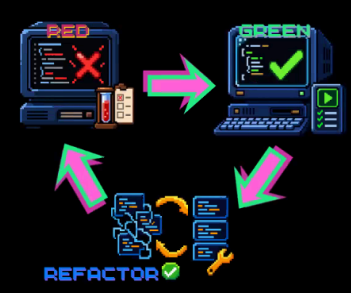



COMIENZA DE NUEVO EL CICLO

RED
➡
GREEN
➡
REFACTOR

```
def test_sumar_dos_numeros_EXITO():
    //Precondiciones
    calculadoraDePrueba = new Calculadora()
    //Pasos
    resultado = calculadoraDePrueba.sumar(a = 3,b = 2)
    //Resultados
    assert(resultado == 5) //nueva comparación a + b

Class Calculadora:
    def sumar(a: Number,b: Number) -> Number:
        return a + b
```



Un ciclo es una pasada en las 3 etapas. Ahora, cambio los parámetros de entrada y por ello tenemos que generalizar el problema, cambiando el código de la suma, no haciendo que me retorne 3. La idea es que en cada ciclo se generalice más la funcionalidad. En este caso, $a + b$ en lugar de 3.



Se proponen una serie de patrones para pasar de una etapa a la otra. Son ayuda-memoria para que el ciclo siga fluyendo y no nos detengamos en el desarrollo.



Ayudan a definir lo que hay que hacer, como:

- Starter test, eligiendo la prueba más trivial, el camino más fácil y obvio para luego compensar el problema y cubrir todas las áreas. Se busca elegir la arista más sencilla para arrancar con el ciclo.
- One step test: se busca seguir con el siguiente en la lineal de complejidad, ir paso a paso para cumplir las metas. Se pueden definir hitos.

- Explanation test: se aplica en una etapa madura del ciclo y permite generar debates sobre el comportamiento esperado del sistema, representando ese debate como un test para sacar las diferencias entre criterios.
- Learning test: cuando quiero conectarme con un sistema externo, librería o algo que no es código propio, definirlo con un test. La mejor forma de aprender es "rompiendo las cosas".
- Another test: es para que la gente no se distraiga. En el momento en quien alguien se aleja del camino, pare, anote la problemática en lista de pendientes y vuelva cuando el test se esté trabajando.
- Regression test: cuando ocurre un problema, defino un test que me permita replicar los pasos que llevaron a la ejecución de ese problema, para que no vuelva a ocurrir en el futuro. Permiten replicar el entorno en el que ocurrió ese error.



- Fake it ('til you make it): hardcodeo el código para que devuelva lo que tiene que devolver la prueba. Por ejemplo, lo hecho con el return 3.
- Triangulación: invita a armar varios casos de falla para que efectivamente asegurar que la falla está en lo que armé en esa aplicación. Se busca generar un problema genérico para todas las alternativas que se plantean. Nos aseguramos de que esté bien/mal.
- Obvious implementation: si ya estamos seguro que la solución es lo que decimos, escribirla tal cual es. En el caso de la suma, podríamos haber escrito directamente $a + b$ para cumplir con la prueba en todos los casos posibles. Se busca escribir el código que resuelva todas las generalidades en PROBLEMAS SENCILLOS.
- One to many: si tengo una función que funciona para colecciones/conjunto de datos, enfocarnos en que funcione mando una unidad y después en iteraciones planteo que se trabaje una colección (para que no solo se sumen 2 números, sino un conjunto de ellos).



- Test list: antes de empezar a desarrollar, hago un listado de las cosas que tengo que realizar.
- Isolated test: es una característica que busca que los patrones sean independientes del resto. No necesito de la ejecución de otro test. Son pruebas unitarias.
- Test first: no escribir código hasta que hayamos escrito una prueba.
- Assert test: cuando cuesta escribir un test por la problemática o su complejidad intrínseca, propone que se arranque desde lo que se quiere probar, primero escribiendo lo que quiero comparar (aserción) Y DESPUES ejecuto o hago una determinada cantidad de pasos. Vamos del final, hasta el inicio.
- Test data: la info. que debemos usar debe ser lo más fácil de entender.
- Evident data: si tengo un cálculo complejo, la relación entre las entradas y las salidas debe ser lo más sencilla posible.



CASO TAXI MOBILE

Ver mapa de taxis

Como Administrador de la Central quiero ver la ubicación de todos los taxis de la central y si tienen viajes en curso para saber la disponibilidad actual

Criterios de Aceptación:

- Se muestran taxis en todos los estados (Verde: Libre, Amarillo: Solicitado, Rojo: Ocupado, Negro: Fuera de Servicio).
- Los taxis que se mostrarán deben tener habilitado el sistema de ubicación.
- Pasando el mouse sobre el icono del auto deben poder visualizarse los datos del viaje si el taxi está ocupado (hora de inicio y costo) y los datos del pasajero (nombre, apellido y número de teléfono celular).
- Si el taxi está solicitado, sólo se visualizarán los datos del pasajero.
- Podrán utilizarse filtros por Barrio, por Estado y por chapa del taxi

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)



Es un caso donde puedo encontrar un auto en una app (uber).

RED

```
def test_mostrar_mapa_taxis_libres_en_barrio_exitito():
    //Precondiciones
    usuarios = [{"user": "salvita", "rol": "Administrador de la Central"}]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    taxis = [{"chapa": "ABC123", "estado": "Libre", "barrio": "Nueva Córdoba", "ubicacion": (10, 20)}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada")
    assert all(taxi.estado == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

def mostrar_mapa(filtros, barrio, chapa, estado):
    pass
```




GREEN

```
def test_mostrar_mapa_taxis_libres_en_barrio_exito():
    //Precondiciones
    usuarios = [{user: "salvita", rol: "Administrador de la Central"}]
    loguear(salvita)
    configurar_ciudad_prueba("Córdoba Capital")
    //carga los datos al sistema.
    taxis = [{'chapa': 'ABC123', 'estado': 'Libre', 'barrio': 'General Paz', 'ubicacion': (10, 20)}]

    //Pasos
    mapa_mostrado = mostrar_mapa(filtros = ["barrio", "estado"], barrio = "General Paz", chapa = "", estado = "Libre")
    //Resultado
    assert all (mapa_mostrado.ciudad.barrio == "General Paz", "No existe tal barrio para esa ciudad ingresada")
    assert all(taxi.barrio == "General Paz" for taxi in mapa_mostrado.taxis, "No todos los taxis están dentro del area filtrada ")
    assert all(taxi.estado.nombre == "Libre" for taxi in mapa_mostrado.taxis, "No todos los taxis están libres")
    assert all(taxi.estado.icono == "verde" for taxi in mapa_mostrado.taxis, "No todos los taxis tienen el icono verde")

    def mostrar_mapa(filtros, barrio, chapa, estado):
        return new Mapa([taxi={chapa = "ABC123", estado = {nombre = "Libre", icono = "verde"}, barrio = "General Paz"}])
```

Se fakeó el resultado a propósito.

REFACTOR

```
def mostrar_mapa(filtros: String[], barrio: Barrio, chapa: String, estado: Estado) -> Mapa:
    estado_libre = new Estado(
        nombre = "Libre",
        icono = "verde"
    )

    barrio_general_paz = new Barrio(
        nombre = "General Paz"
    )

    taxi = new Taxi(
        chapa = "ABC123",
        estado = estado_libre,
        barrio = barrio_general_paz
    )

    taxis = [taxi]

    return new Mapa(taxis = taxis)
```

Acá se crea un código legible, propiamente dicho.

¿CÓMO SIGUE?

1. Volver a iniciar el ciclo.
2. Considerar el resto de casos de pruebas, añadiendo las alternativas planteadas en la user.
3. Refactorizar.

Hacer TDD sin refactorizar es simplemente escribir código malo con pruebas.



Sin refactor, no existe TDD.

EJEMPLO FINAL

def mostrar_mapa(filtros: Filtros, barrio: Barrio, chapa: String, estado: Estado) -> Mapa:

// Caso de prueba taxis desconectados

```
if (filtros== 'Mostrar todos los taxis')
  taxis = buscarTaxiEnBarrio(barrio)
  (ubicación == None) //ubicación desconectada
  taxis = buscarTaxiDesconectado (ubicacion)
```

// Caso de prueba mostrar taxis en distintos estados en un barrio

```
if (filtros== 'Mostrar taxis por estado y por barrio')
  taxis = buscarTaxiPorEstadoYPorBarrio (barrio, estado)
```

// Caso de prueba mostrar taxis por chapa

```
if (filtros == 'Mostrar taxis por chapa')
  taxis = buscarTaxiPorChapa (chapa)
```

// Caso de prueba mostrar taxis por barrio y datos de viaje

```
if (filtros== 'Mostrar taxis por barrio')
  taxisFiltrados = buscarTaxiEnBarrio (barrio)
  //foreach simplificado
  if (taxis[i].getEstado() == 'Solicitado' && filtros== 'Mostrar datos viaje')
    taxis =
      taxis[i].getChapa()
      taxis[i].getViajeActual().getPasajero().getNombre()
      taxis[i].getViajeActual().getPasajero().getTelefono()
      taxis[i].getViajeActual().getFechaYHora()
```

return new Mapa(taxis=taxis)

Pruebas de Usuario

- Probar visualizar un conjunto de taxis libres (pasa)
- Probar visualizar un conjunto de taxis solicitados (pasa)
- Probar visualizar un conjunto de taxis fuera de servicio (pasa)
- Probar visualizar un conjunto de taxis ocupados (pasa)
- Probar visualizar un conjunto de taxis libres, solicitados, fuera de servicio y ocupados (pasa)
- Probar visualizar un taxi conectado al sistema por número de chapa (pasa)
- Probar visualizar en el mapa un taxi no conectado al sistema de ubicación (falla)
- Probar visualizar un conjunto de taxis en un Barrio inexistente (falla)

